Python Programming – Student Learning Guide

VPro Skills EduTech · www.vproskills.com

This guide introduces the core concepts of Python programming in simple language, with practical code examples for every topic. Read each section, then try the examples yourself in the Python shell.

Contents

- 1. Introduction to Python
- 2. Features of Python
- 3. Python Installation
- 4. Python Variables
- 5. Python Data Types
- 6. Strings
- 7. Lists
- 8. Tuples
- 9. Dictionaries
- 10. Sets
- 11. Conditional Statements
- 12. Loops
- 13. Functions
- 14. Object-Oriented Programming
- 15. Exception Handling

1. Introduction to Python

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. It is designed to be easy to read and easy to write. Python code looks close to plain English, which is why it is one of the most popular first languages for students.

Python is an interpreted language. This means you do not need to compile your program before running it. The Python interpreter reads your code line by line and executes it immediately. This makes it quick to test small ideas and learn by experimenting.

Where is Python used?

- Web development (Django, Flask, FastAPI)
- Data analysis and visualisation (pandas, Matplotlib)
- Artificial Intelligence and Machine Learning (scikit-learn, TensorFlow, PyTorch)
- Automation and scripting of everyday tasks
- Software testing, desktop applications and games

Your first Python program

```
print("Hello, World!")
# Output: Hello, World!
```

The print() function displays text on the screen. Lines that start with # are comments. Python ignores comments; they are notes written for humans who read the code.

2. Features of Python

Python has several features that make it popular with beginners and professionals alike.

- **Simple and readable syntax:** Python uses indentation (spaces) instead of curly braces to group code, which keeps programs clean and readable.
- **Interpreted:** Code runs line by line, so errors are reported where they happen.
- **Dynamically typed:** You do not declare variable types. Python works out the type when the program runs.
- **Free and open source:** Anyone can download, use and share Python at no cost.
- **Cross-platform:** The same Python program runs on Windows, macOS and Linux.
- **Large standard library:** Python comes with ready-made modules for files, dates, maths, networking and much more – often described as 'batteries included'.
- **Object-oriented:** Python supports classes and objects, but you can also write simple procedural code.
- **Huge community:** Thousands of third-party packages are available through pip.

Example: readable syntax

```
marks = 82
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

3. Python Installation

To start programming, install Python 3 from the official website python.org. Always choose the latest stable Python 3 version.

Installing on Windows

- Download the Windows installer from python.org/downloads.
- Run the installer and tick the box **'Add Python to PATH'** before clicking Install Now.
- Open Command Prompt and type `python --version` to confirm.

Installing on macOS

- Download the macOS installer from python.org/downloads and run it, or use Homebrew: `brew install python`.
- Open Terminal and type `python3 --version` to confirm.

Running Python code

You can run Python in two ways. The **interactive shell** (type python or python3 in a terminal) runs one line at a time and is great for quick experiments. A **script file** ends with .py and is run with a command such as:

```
python hello.py      # Windows
python3 hello.py     # macOS / Linux
```

pip is Python's package installer. It downloads extra libraries, for example: `pip install requests`.

4. Python Variables

A variable is a name that stores a value in memory so you can use it later. In Python you create a variable simply by assigning a value with the = sign. There is no need to declare its type first.

```
student_name = "Ravi"
age = 20
height = 5.8
is_enrolled = True
```

Rules for variable names

- A name must start with a letter or an underscore (_), not a number.

- It can contain only letters, numbers and underscores.
- Names are case-sensitive: age, Age and AGE are three different variables.
- Python keywords such as if, for, class and def cannot be used as names.

Multiple assignment and dynamic typing

```
x, y, z = 1, 2, 3
a = b = 0

value = 10        # value is an int
value = "ten"     # now value is a str
```

Because Python is dynamically typed, the same variable can later hold a value of a different type. Use the type() function to check the type of any value: `type(age)` returns <class 'int'>.

5. Python Data Types

A data type tells Python what kind of value a variable holds and what operations are allowed on it. The main built-in data types are:

- **int** – whole numbers, for example 10, -3, 2024
- **float** – decimal numbers, for example 3.14, -0.5
- **complex** – complex numbers, for example 2 + 3j
- **str** – text, for example "Python"
- **bool** – True or False
- **list** – ordered, changeable collection: [1, 2, 3]
- **tuple** – ordered, unchangeable collection: (1, 2, 3)
- **dict** – key-value pairs: {"name": "Asha"}
- **set** – unordered collection of unique items: {1, 2, 3}
- **NoneType** – the special value None, meaning 'no value'

Type conversion

You can convert between types using functions such as int(), float(), str() and bool(). This is called type casting.

```
age_text = "21"
age = int(age_text)      # 21 (int)
price = float("99.5")    # 99.5
label = str(100)         # "100"
print(type(price))       # <class 'float'>
```

Data types can be grouped as **mutable** (can be changed after creation: list, dict, set) and **immutable** (cannot be changed: int, float, str, tuple, bool).

6. Strings

A string is a sequence of characters enclosed in single quotes, double quotes or triple quotes. Strings are immutable, which means their characters cannot be changed in place.

```
course = "Python"
message = 'Welcome to VPro Skills'
paragraph = """This string
spans multiple lines."""
```

Indexing and slicing

Each character has a position called an index, starting at 0. Negative indexes count from the end. Slicing extracts part of a string using [start:end].

```
word = "Python"
print(word[0])     # P
print(word[-1])    # n
print(word[0:3])   # Pyt
print(len(word))   # 6
```

Common string methods

```
name = "  vpro skills  "
print(name.strip())          # "vpro skills"
print(name.upper())          # "  VPRO SKILLS  "
print("a,b,c".split(","))    # ['a', 'b', 'c']
print("Hi Ravi".replace("Ravi", "Asha"))
```

f-strings

f-strings are the easiest way to put variables inside text.

```
name = "Asha"
score = 95
print(f"{name} scored {score} marks")
```

7. Lists

A list is an ordered, mutable collection of items written inside square brackets. A list can hold values of different types and can grow or shrink while the program runs.

```
fruits = ["apple", "banana", "mango"]
mixed = [1, "two", 3.0, True]
```

Accessing and changing items

```
print(fruits[0])        # apple
fruits[1] = "grapes"    # lists are mutable
print(fruits)           # ['apple', 'grapes', 'mango']
```

Useful list methods

- append(x) – add an item to the end
- insert(i, x) – add an item at position i
- remove(x) – remove the first matching item
- pop() – remove and return the last item
- sort() – sort the list in place
- len(list) – number of items

```
numbers = [5, 2, 9]
numbers.append(1)
numbers.sort()
print(numbers)          # [1, 2, 5, 9]
```

List comprehension

A list comprehension builds a new list in a single readable line.

```
squares = [n * n for n in range(1, 6)]
print(squares)          # [1, 4, 9, 16, 25]
```

8. Tuples

A tuple is an ordered collection like a list, but it is **immutable**: once created, its items cannot be added, removed or changed. Tuples are written with round brackets.

```
point = (10, 20)
colours = ("red", "green", "blue")
single = ("only",)     # a one-item tuple needs a comma
```

Tuples support indexing and slicing just like lists, and they are slightly faster and use less memory. Use a tuple for data that should not change, such as coordinates or the days of the week.

```
x, y = point          # tuple unpacking
print(x, y)           # 10 20
# point[0] = 5  ->  TypeError: 'tuple' object does not support item assignment
```

Difference between List and Tuple

- **Syntax:** a list uses square brackets [ ]; a tuple uses round brackets ( ).
- **Mutability:** a list is mutable (can be changed); a tuple is immutable (cannot be changed).
- **Methods:** lists have many methods (append, remove, sort); tuples only have count() and index().
- **Performance:** tuples are faster and use less memory than lists.
- **Use case:** use a list for data that changes; use a tuple for fixed data. Tuples can also be used as dictionary keys, lists cannot.

9. Dictionaries

A dictionary stores data as **key-value pairs** inside curly braces. Each key must be unique and immutable (for example a string or number). Dictionaries are mutable and keep items in the order they were inserted.

```
student = {
    "name": "Ravi",
    "age": 20,
    "course": "Python"
}
```

Reading, adding and removing values

```
print(student["name"])            # Ravi
print(student.get("city", "N/A")) # N/A (safe access)
student["city"] = "Hyderabad"     # add a new key
student["age"] = 21               # update a value
del student["course"]             # remove a key
```

Looping through a dictionary

```
for key, value in student.items():
    print(key, "->", value)
```

Other helpful methods are keys(), values(), items(), update() and pop(). Dictionaries are ideal when you want to look up information by a meaningful name instead of a position.

10. Sets

A set is an unordered collection of **unique** items written inside curly braces. Duplicate values are removed automatically. Sets are mutable, but their items must be immutable, and items cannot be accessed by index.

```
numbers = {1, 2, 3, 3, 2}
print(numbers)      # {1, 2, 3}
empty = set()       # {} creates an empty dict, not a set
```

Set operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a | b)   # union        {1, 2, 3, 4, 5}
print(a & b)   # intersection {3, 4}
print(a - b)   # difference   {1, 2}
print(a ^ b)   # symmetric difference {1, 2, 5}
```

Use add() to insert an item and remove() or discard() to delete one. A common practical use of a set is removing duplicates from a list: `unique = list(set(items))`.

11. Conditional Statements

Conditional statements let a program make decisions. Python uses if, elif (else if) and else. The code that belongs to each branch must be indented.

```
marks = 72

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"

print("Grade:", grade)   # Grade: C
```

Comparison and logical operators

- Comparison: == (equal), != (not equal), >, <, >=, <=
- Logical: and (both true), or (at least one true), not (reverses the result)

```
age = 19
has_id = True
if age >= 18 and has_id:
    print("Entry allowed")
```

Conditional expression (ternary)

```
status = "Adult" if age >= 18 else "Minor"
```

12. Loops

Loops repeat a block of code. Python has two loops: the **for** loop and the **while** loop.

for loop

A for loop goes through each item of a sequence such as a list, string or range.

```
for i in range(1, 6):
    print(i)            # 1 2 3 4 5

for fruit in ["apple", "mango"]:
    print(fruit)
```

while loop

A while loop keeps running as long as its condition is True. Make sure the condition eventually becomes False, or the loop will run forever.

```
count = 1
while count <= 3:
    print("Count:", count)
    count += 1
```

break, continue and pass

- **break** stops the loop immediately.
- **continue** skips the rest of the current iteration and moves to the next one.
- **pass** does nothing; it is a placeholder where code is required.

```
for n in range(10):
    if n == 5:
        break
    if n % 2 == 0:
        continue
    print(n)            # 1 3
```

13. Functions

A function is a reusable block of code that performs a specific task. Functions help you avoid repeating code and make programs easier to read and test. Define a function with the **def** keyword.

```
def greet(name):
    """Return a greeting message."""
    return f"Hello, {name}!"

print(greet("Asha"))     # Hello, Asha!
```

Parameters and arguments

- **Parameters** are the names listed in the function definition.
- **Arguments** are the actual values passed when calling the function.
- **Default parameters** provide a value if none is given.
- **Keyword arguments** are passed by name, so order does not matter.

```
def total_price(price, quantity=1, discount=0):
    return price * quantity - discount

print(total_price(100))                    # 100
print(total_price(100, quantity=3, discount=50))  # 250
```

*args, **kwargs and lambda

```
def add_all(*numbers):
    return sum(numbers)

print(add_all(1, 2, 3))   # 6

square = lambda x: x * x  # a small anonymous function
print(square(4))          # 16
```

The return statement sends a value back to the caller. A function without return gives back None.

14. Object-Oriented Programming

Object-Oriented Programming (OOP, often called OOPS) is a way of organising code around **objects**. A **class** is a blueprint, and an **object** is an instance created from that blueprint. Objects combine data (attributes) and behaviour (methods).

```
class Student:
    def __init__(self, name, course):
        self.name = name          # attribute
        self.course = course

    def introduce(self):          # method
        return f"I am {self.name}, learning {self.course}."

ravi = Student("Ravi", "Python")
print(ravi.introduce())
```

__init__ is the constructor; it runs automatically when an object is created. self refers to the current object.

The four pillars of OOP

- **Encapsulation:** bundling data and methods together and hiding internal details (for example using a leading underscore, _balance).
- **Inheritance:** a child class reuses and extends the code of a parent class.
- **Polymorphism:** different classes provide their own version of the same method name.
- **Abstraction:** showing only essential features and hiding complex implementation.

Inheritance example

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):              # overriding = polymorphism
        return "Woof!"

print(Dog().speak())   # Woof!
```

15. Exception Handling

An exception is an error that happens while a program is running, such as dividing by zero or opening a missing file. Without handling, the program stops with an error message. Exception handling lets the program respond gracefully instead of crashing.

try, except, else and finally

```
try:
    number = int(input("Enter a number: "))
    result = 100 / number
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print("Result:", result)      # runs only if no error
finally:
    print("Done.")                # always runs
```

Common built-in exceptions

- ValueError – a value has the right type but an invalid content
- TypeError – an operation is used on the wrong type
- ZeroDivisionError – division by zero
- IndexError – a list index is out of range
- KeyError – a dictionary key does not exist
- FileNotFoundError – a file cannot be found

Raising your own exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age
```

Good practice: catch specific exceptions rather than using a bare except, and keep the code inside try blocks as small as possible.